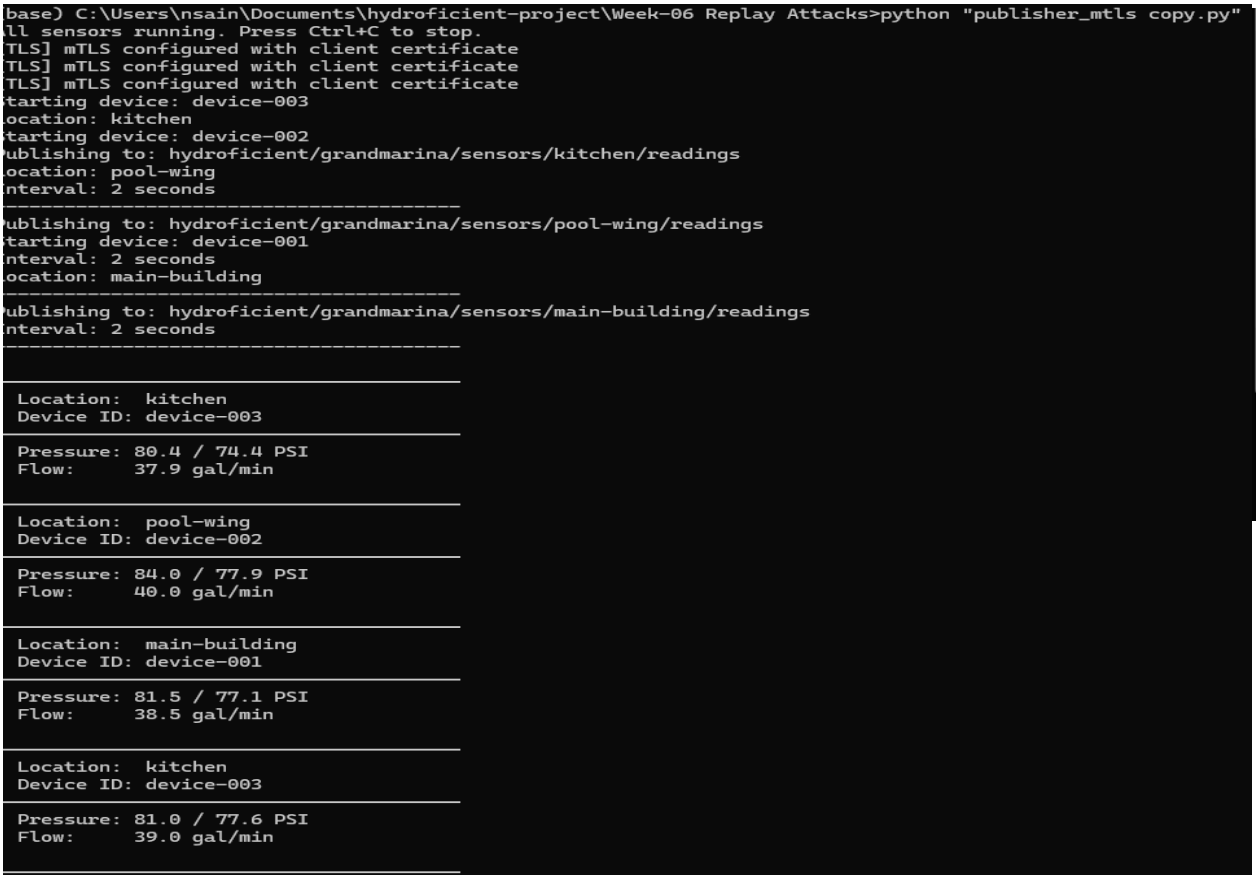
Navdeep Saini
Week 6 - Replay Attacks

Focusing Week 6

Replay Attack Simulation

- This first step we will take is to see where the weakness lies within our publisher and subscriber code. So far I know that the on_message function within our subscriber file has no verification. This means that messages are being sent freely and can be snatched through network traffic and replayed at a later time which can cause a lot of issues.
    - This first test is to just see whether logs are being denied from being captured or not. I started my mosquitto_mtls, subscriber.py, and publisher.py file. Now it's time to capture logs running the replay_attack.py file which will run the simulation while our publisher is online. `python replay_attacker.py --mode capture --count 5`



```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-06 Replay Attacks>python replay_attacker.py --mode capture --c
ount 5
============================================================
REPLAY ATTACK TOOL - Capture Mode
============================================================
Target: 5 messages
Output: captured_messages.json
============================================================
C:\Users\nsain\Documents\hydroficient-project\Week-06 Replay Attacks\replay_attacker.py:120: DeprecationWarning: Callbac
k API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="replay-attacker-capture", **MQTT_CLIENT_ARGS)

[CONNECTING] localhost:8883...
[LISTENING] Waiting for messages to capture...

[CONNECTED] Listening on: hydroficient/grandmarina/#
[CAPTURED 1/5] device-003 - N/A LPM
[CAPTURED 2/5] device-002 - N/A LPM
[CAPTURED 3/5] device-001 - N/A LPM
[CAPTURED 4/5] device-003 - N/A LPM
[CAPTURED 5/5] device-002 - N/A LPM

[DONE] Captured 5 messages
```

This is how simple it is, the attacker listens on port 8883 and waits for messages to be captured on the HYDROLOGIC devices, it's also subscribed to everything within The Grand Marina using the # wildcard which subscribes to everything.

```
C:\Users\nsain\Documents\hydroficient-project\Week-06 Replay Attacks\replay_attacker.py:181: DeprecationWarning: Callbac
k API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="replay-attacker", **MQTT_CLIENT_ARGS)

[CONNECTING] localhost:8883...

[REPLAYING] Sending captured messages...

[REPLAYING 1/6] Topic: hydroficient/grandmarina/sensors/kitchen/readings
  Originally captured: 2026-04-27T22:49:59.812270Z
  Replayed at: 2026-04-27T22:59:10.990237Z

[REPLAYING 2/6] Topic: hydroficient/grandmarina/sensors/pool-wing/readings
  Originally captured: 2026-04-27T22:49:59.812616Z
  Replayed at: 2026-04-27T22:59:11.991264Z

[REPLAYING 3/6] Topic: hydroficient/grandmarina/sensors/main-building/readings
  Originally captured: 2026-04-27T22:49:59.812729Z
  Replayed at: 2026-04-27T22:59:12.993729Z

[REPLAYING 4/6] Topic: hydroficient/grandmarina/sensors/kitchen/readings
  Originally captured: 2026-04-27T22:50:01.813381Z
  Replayed at: 2026-04-27T22:59:13.995776Z

[REPLAYING 5/6] Topic: hydroficient/grandmarina/sensors/pool-wing/readings
  Originally captured: 2026-04-27T22:50:01.813813Z
  Replayed at: 2026-04-27T22:59:14.997399Z

[REPLAYING 6/6] Topic: hydroficient/grandmarina/sensors/main-building/readings
  Originally captured: 2026-04-27T22:50:01.814146Z
  Replayed at: 2026-04-27T22:59:15.998158Z

[DONE] Replayed 6 messages
[NOTE] Check your subscriber - did it accept them all?
```

Above I ran a replay command to see what was captured during the listening phase of the attack.

The Three Defenses: Timestamp validation, sequence counters, and HMAC signing

To protect against replay attacks I have implemented three defense methods to add security to our data. The three will be listed below and given an explanation.

Timestamp Validation

- Timestamp validation adds an expiration date on a given log that was generated and sent, this allows us to verify if the log has been duplicated within the given window which for this example is 30 seconds, this makes sense since our log is generated every 2-5 seconds. This code has been implemented into the publisher file as that is where the verification will take place.

```
@staticmethod
def check_timestamp(message):

    msg_time = datetime.fromisoformat(message["timestamp"].replace("Z", "+00:00"))
    now = datetime.now(timezone.utc)
    age = ((now - msg_time).total_seconds())

    if age <= MAX_AGE_SECONDS:
        return True  # Fresh reading
    else:
        return False # Stale reading, possible replay attack
```

Sequence Counter

- The sequence counter was implemented into both the publisher & subscriber files, this counter is an upgraded version of our original counter but with this it gives each log a number that is later verified to see if a replay has happened.

(subscriber file)

```
def check_sequence(message):
    device_id = message["device_id"]
    sequence = message["sequence"]

    last_seen = device_counters.get(device_id, 0)

    if sequence > last_seen:
        device_counters[device_id] = sequence
        return True
    else:
        return False
```

(publisher file)

```python
def get_reading(self):
    """Generate a sensor reading with realistic variation."""
    self.counter += 1
    return {
        "device_id": self.device_id,  # identity
        "location": self.location,    # context (optional but recommended)
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "sequence": self.counter,  # Add sequence number for replay attack simulation
        "pressure_upstream": round(self.base_pressure_up + random.uniform(-2, 2), 1),
        "pressure_downstream": round(self.base_pressure_down + random.uniform(-2, 2), 1),
        "flow_rate": round(self.base_flow + random.uniform(-3, 3), 1),
    }
```

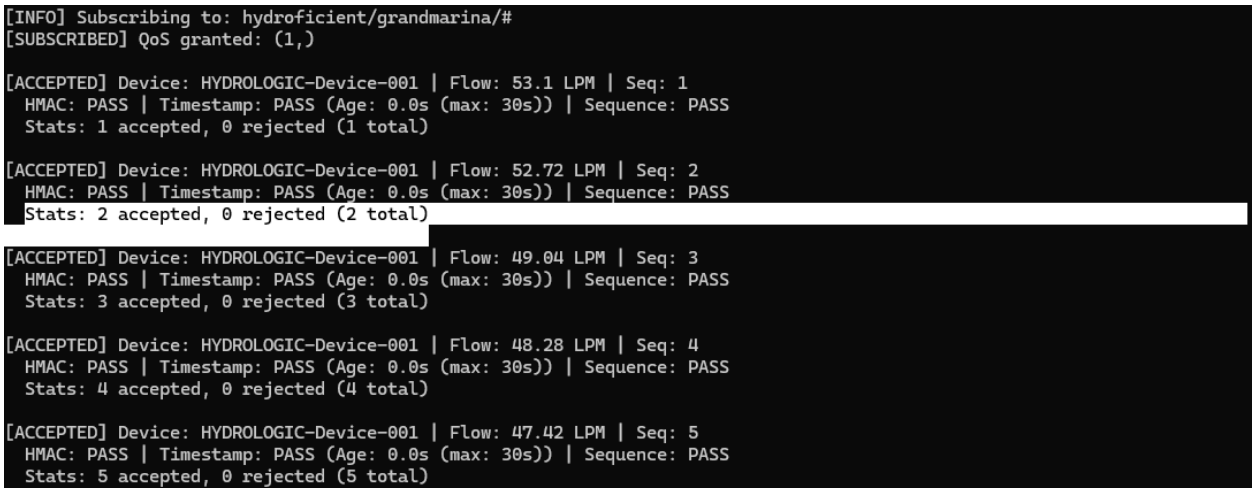
HMAC Signature

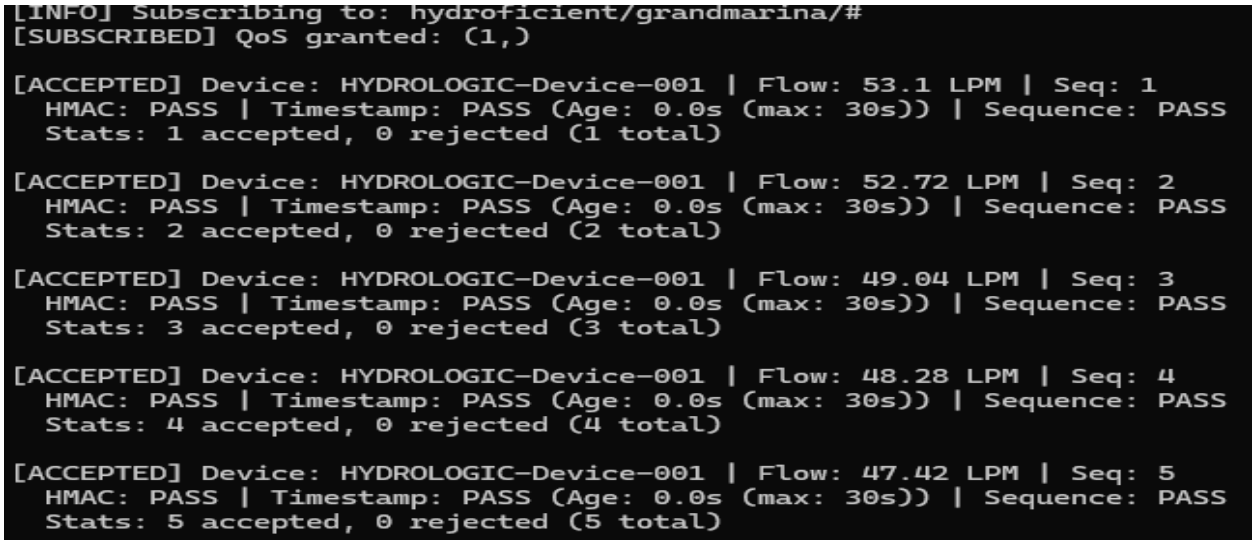
- HMAC Signatures make each log tamper proof, this will allow us to see if anything has been changed in transit of a log to detect a potential replay attack. This was implemented into the publisher file as that is where data logs will be processed and sent out to the dashboard.

```python
@staticmethod
def compute_hmac(message_dict):
    # Create a copy of the message without the HMAC field
    msg_copy = {k: v for k, v in message_dict.items() if k != "hmac"}

    # This sorts keys for consistent HMAC generation
    msg_string = json.dumps(msg_copy, sort_keys=True)

    # Now this signs with shared key

    signature = hmac.new(
        SHARED_SECRET.encode("utf-8"),
        msg_string.encode("UTF-8"),
        hashlib.sha256
    ).hexdigest()
    return signature
```

The Output

- After successfully implementing a sequence counter, timestamp validation, and a HMAC signature the output was successful, each new log reading was given a sequence number, expiration time window, and a unique signature!





Replay Attack #2

- Before we tested a replay attack without any defenses, just plain out in the open to steal. Now I ran another replay attack, and the results are impressive! With the implementation of sequence counters and an expiration time, anything within the expiration window will be detected and rejected and the same goes for when it's outside of the window.

```
C:\Users\nsain\Documents\hydroficient-project\Week-06 Repla
k API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="replay-attacker-capture",

[CONNECTING] localhost:8883...
[LISTENING] Waiting for messages to capture...

[CONNECTED] Listening on: hydroficient/grandmarina/#
[CAPTURED 1/5] HYDROLOGIC-Device-001 - 48.47 LPM
[CAPTURED 2/5] HYDROLOGIC-Device-001 - 54.82 LPM
[CAPTURED 3/5] HYDROLOGIC-Device-001 - 48.29 LPM
[CAPTURED 4/5] HYDROLOGIC-Device-001 - 46.33 LPM
[CAPTURED 5/5] HYDROLOGIC-Device-001 - 46.67 LPM

[DONE] Captured 5 messages
[SAVED] captured_messages.json
[SAVED] 5 messages saved to captured_messages.json
```

```
[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 48.47 LPM | Seq: 9
  Failed check: TIMESTAMP
  Reason: Age: 61.3s (max: 30s)
  Stats: 20 accepted, 1 rejected (21 total)

[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 54.82 LPM | Seq: 10
  Failed check: TIMESTAMP
  Reason: Age: 57.3s (max: 30s)
  Stats: 20 accepted, 2 rejected (22 total)

[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 48.29 LPM | Seq: 11
  Failed check: TIMESTAMP
  Reason: Age: 53.3s (max: 30s)
  Stats: 20 accepted, 3 rejected (23 total)

[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 46.33 LPM | Seq: 12
  Failed check: TIMESTAMP
  Reason: Age: 49.3s (max: 30s)
  Stats: 20 accepted, 4 rejected (24 total)

[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 52.74 LPM | Seq: 22
  HMAC: PASS | Timestamp: PASS (Age: 0.0s (max: 30s)) | Sequence: PASS
  Stats: 21 accepted, 4 rejected (25 total)

[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 46.67 LPM | Seq: 13
  Failed check: TIMESTAMP
  Reason: Age: 45.3s (max: 30s)
  Stats: 21 accepted, 5 rejected (26 total)
```

Twelve Experiments

- These next experiments will cover everything in depth to see which works and doesn't work, we will check three defenses which is a total of twelve experiments, 4 being for 1 each. Configurations that will be tested separately are none (no validation), timestamp, counter, and all (timestamp + counter + HMAC). Dependent variables will display a rejection rate of 0% if the attack was successful and 100% if the attack was blocked. Everything else stays the same, meaning broker configuration, mTLS certificates, and the number of test messages which is limited to 5 per experiment.

Attack Types

- The following three attack types go into depth on what I will be conducting.
  - **Immediate replay** sends captured messages within 5 seconds of the original. This tests whether defenses can catch replays that are still "fresh" — the timestamp hasn't expired yet. This is the trickiest attack to defend against because the message looks recent.
  - **Delayed replay** sends captured messages after 60 seconds. This simulates the most common real-world scenario — an attacker records messages and replays them minutes or hours later. Timestamp validation should catch this easily.
  - **Modified replay** sends captured messages with altered sensor values (specifically, setting `flow_rate` to 0.0 to simulate a shutoff command). This tests whether defenses can detect tampering. The attacker isn't just replaying — they're changing the data to cause specific harm.

Expected Outcome

- Below lists the expected outcome for each experiment from a 0% rejection rate to 100%.
  - **Experiment 1** (No Defense): having no validation the rejection rate should be at 0%.
  - **Experiment 2** (Timestamp only): Delayed replays should be caught that are outside of the expiration window (30 seconds). Anything that is an immediate replay should be caught within the expiration window. Modified replays are going to appear valid if the data is changed.
  - **Experiment 3** (Counter only): Any replay that is immediate or delayed will be rejected because of the counter giving the data a sequence number. Modified replays are still harder to reject as it can change the sequence number to a higher value.
  - **Experiment 4** (All Defenses): Everything should be rejected 100% across all attack types due to having timestamps, counters, and hmac signatures enabled for defense.

NOTE: **I will only be displaying the final results of each experiment and 1 attack type as showing all would take up a lot of space.**

Experiment 1 (No Defense)

Rejection Rate 0% all around

```
============================================================
EXPERIMENT: Defense=none vs Attack=modified
============================================================

[STEP 1] Generating 5 legitimate messages...
  Message 1: seq=1, flow=50.75 LPM
  Message 2: seq=2, flow=49.96 LPM
  Message 3: seq=3, flow=54.0 LPM
  Message 4: seq=4, flow=46.33 LPM
  Message 5: seq=5, flow=48.68 LPM

[STEP 2] Processing legitimate messages through subscriber...
  [ACCEPTED] seq=1 - No defenses active
  [ACCEPTED] seq=2 - No defenses active
  [ACCEPTED] seq=3 - No defenses active
  [ACCEPTED] seq=4 - No defenses active
  [ACCEPTED] seq=5 - No defenses active

[STEP 3] Creating modified replay attack...
  Replaying 5 MODIFIED messages

[STEP 4] Running attack...
  [ACCEPTED] seq=6 - No defenses active
  [ACCEPTED] seq=7 - No defenses active
  [ACCEPTED] seq=8 - No defenses active
  [ACCEPTED] seq=9 - No defenses active
  [ACCEPTED] seq=10 - No defenses active

[RESULTS]
  Messages tested: 5
  Accepted: 5
  Rejected: 0
  Rejection rate: 0%

============================================================
EXPERIMENT SUMMARY
============================================================
Defense      Attack       Accepted   Rejected   Rate
------------------------------------------------------------
none         immediate    5          0          0%
none         delayed      5          0          0%
none         modified     5          0          0%
```

Experiment 2 (Timestamp Only)

Rejection rate 0% immediate, 100% delayed, 0% modified

```
============================================================
EXPERIMENT: Defense=timestamp vs Attack=delayed
============================================================

[STEP 1] Generating 5 legitimate messages...
  Message 1: seq=1, flow=49.96 LPM
  Message 2: seq=2, flow=53.18 LPM
  Message 3: seq=3, flow=47.29 LPM
  Message 4: seq=4, flow=45.99 LPM
  Message 5: seq=5, flow=52.7 LPM

[STEP 2] Processing legitimate messages through subscriber...
  [ACCEPTED] seq=1 - All checks passed
  [ACCEPTED] seq=2 - All checks passed
  [ACCEPTED] seq=3 - All checks passed
  [ACCEPTED] seq=4 - All checks passed
  [ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating delayed replay attack...
  Replaying 5 messages (simulating 60s delay)

[STEP 4] Running attack...
  [REJECTED] seq=1 - Message too old (60.0s > 30s)
  [REJECTED] seq=2 - Message too old (60.0s > 30s)
  [REJECTED] seq=3 - Message too old (60.0s > 30s)
  [REJECTED] seq=4 - Message too old (60.0s > 30s)
  [REJECTED] seq=5 - Message too old (60.0s > 30s)

[RESULTS]
  Messages tested: 5
  Accepted: 0
  Rejected: 5
  Rejection rate: 100%
============================================================
EXPERIMENT SUMMARY
============================================================
Defense      Attack       Accepted   Rejected   Rate
------------------------------------------------------
timestamp    immediate    5          0          0%
timestamp    delayed      0          5          100%
timestamp    modified     5          0          0%

[SAVED] Results saved to experiment_results.json
```

Experiment 3 (Counter Only)

Rejection rate 100% immediate, 100% delayed, 0% modified

```
============================================================
EXPERIMENT: Defense=counter vs Attack=immediate
============================================================

[STEP 1] Generating 5 legitimate messages...
  Message 1: seq=1, flow=50.5 LPM
  Message 2: seq=2, flow=50.78 LPM
  Message 3: seq=3, flow=53.53 LPM
  Message 4: seq=4, flow=52.23 LPM
  Message 5: seq=5, flow=45.86 LPM

[STEP 2] Processing legitimate messages through subscriber...
  [ACCEPTED] seq=1 - All checks passed
  [ACCEPTED] seq=2 - All checks passed
  [ACCEPTED] seq=3 - All checks passed
  [ACCEPTED] seq=4 - All checks passed
  [ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating immediate replay attack...
  Replaying 5 messages (2s delay)

[STEP 4] Running attack...
  [REJECTED] seq=1 - Sequence 1 <= last seen 5
  [REJECTED] seq=2 - Sequence 2 <= last seen 5
  [REJECTED] seq=3 - Sequence 3 <= last seen 5
  [REJECTED] seq=4 - Sequence 4 <= last seen 5
  [REJECTED] seq=5 - Sequence 5 <= last seen 5

[RESULTS]
  Messages tested: 5
  Accepted: 0
  Rejected: 5
  Rejection rate: 100%
============================================================
EXPERIMENT SUMMARY
============================================================
Defense      Attack       Accepted   Rejected   Rate
------------------------------------------------------------
counter      immediate    0          5          100%
counter      delayed      0          5          100%
counter      modified     5          0          0%
```

Experiment 3 (Timestamp, Counter, HMAC)

Rejection rate 100% immediate, 100% delayed, 100% modified

```
============================================================
EXPERIMENT: Defense=all vs Attack=modified
============================================================

[STEP 1] Generating 5 legitimate messages...
  Message 1: seq=1, flow=46.09 LPM
  Message 2: seq=2, flow=46.88 LPM
  Message 3: seq=3, flow=46.23 LPM
  Message 4: seq=4, flow=50.46 LPM
  Message 5: seq=5, flow=46.31 LPM

[STEP 2] Processing legitimate messages through subscriber...
  [ACCEPTED] seq=1 - All checks passed
  [ACCEPTED] seq=2 - All checks passed
  [ACCEPTED] seq=3 - All checks passed
  [ACCEPTED] seq=4 - All checks passed
  [ACCEPTED] seq=5 - All checks passed

[STEP 3] Creating modified replay attack...
  Replaying 5 MODIFIED messages

[STEP 4] Running attack...
  [REJECTED] seq=6 - HMAC mismatch
  [REJECTED] seq=7 - HMAC mismatch
  [REJECTED] seq=8 - HMAC mismatch
  [REJECTED] seq=9 - HMAC mismatch
  [REJECTED] seq=10 - HMAC mismatch

[RESULTS]
  Messages tested: 5
  Accepted: 0
  Rejected: 5
  Rejection rate: 100%
============================================================
EXPERIMENT SUMMARY
============================================================
Defense      Attack       Accepted   Rejected   Rate
------------------------------------------------------------
all          immediate    0          5          100%
all          delayed      0          5          100%
all          modified     0          5          100%
```

HYDROLOGIC SYSTEM GRAPH